

认识Vue3

1. Vue2 选项式 API vs Vue3 组合式API

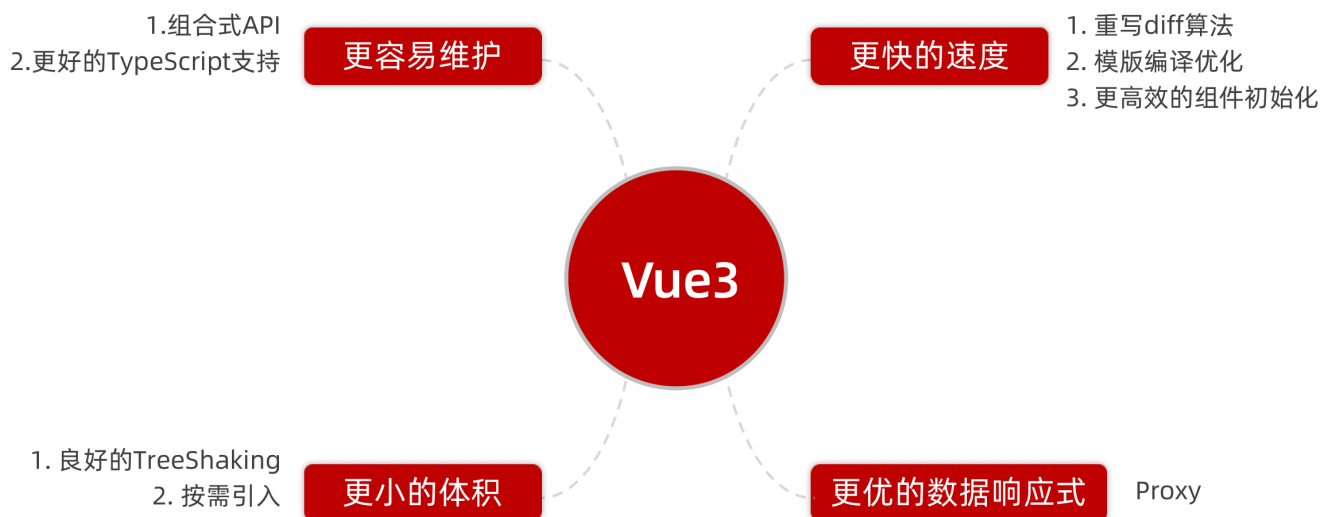
```
<script>
export default {
  data(){
    return {
      count:0
    }
  },
  methods:{
    addCount(){
      this.count++
    }
  }
}
</script>
```

```
<script setup>
import { ref } from 'vue'
const count = ref(0)
const addCount = ()=> count.value++
</script>
```

特点:

1. 代码量变少
2. 分散式维护变成集中式维护

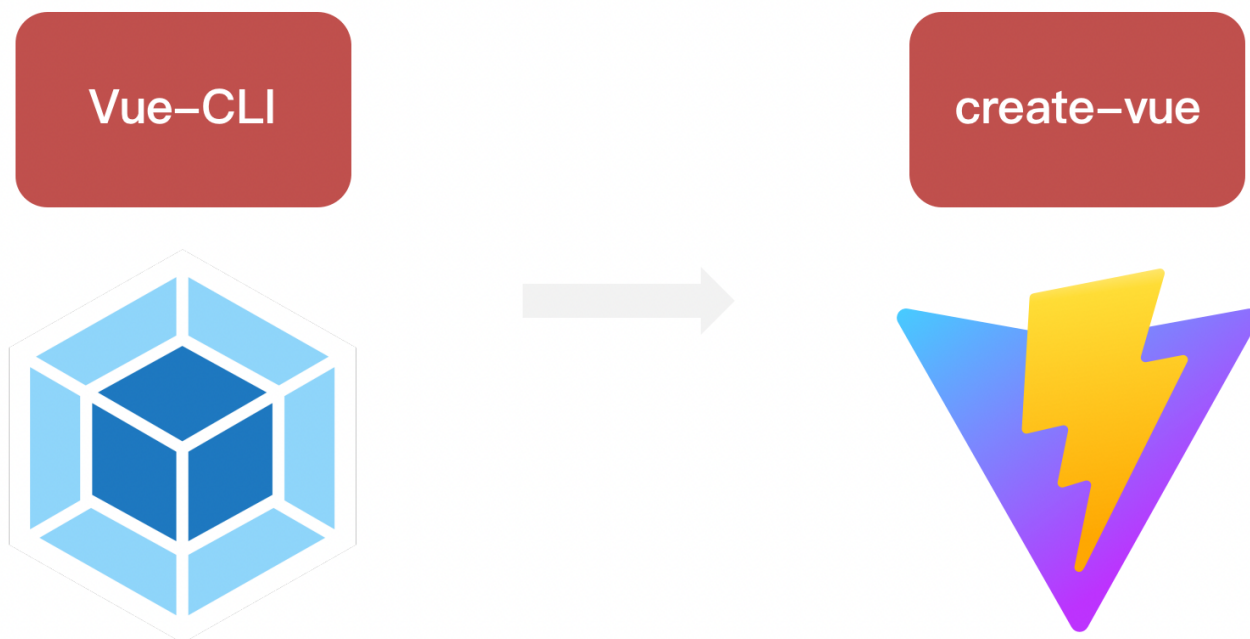
2. Vue3的优势



使用create-vue搭建Vue3项目

1. 认识create-vue

create-vue是Vue官方新的脚手架工具，底层切换到了 vite（下一代前端工具链），为开发提供极速响应



2. 使用create-vue创建项目

前置条件 - 已安装16.0或更高版本的Node.js

执行如下命令，这一指令将会安装并执行 create-vue

```
npm init vue@latest
```

```
chaipeng@localhost pro % npm init vue@latest

Vue.js - The Progressive JavaScript Framework

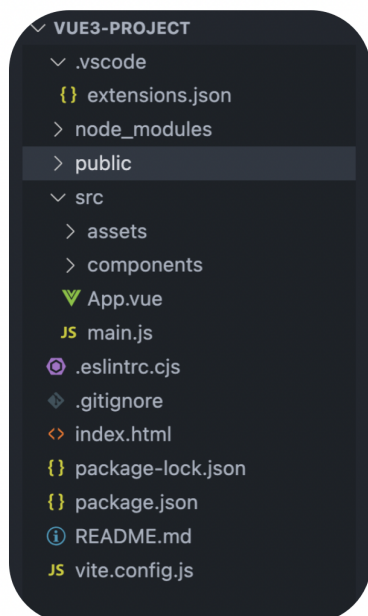
✓ Project name: ... vue3-project
✓ Add TypeScript? ... No / Yes
✓ Add JSX Support? ... No / Yes
✓ Add Vue Router for Single Page Application development? ... No / Yes
✓ Add Pinia for state management? ... No / Yes
✓ Add Vitest for Unit Testing? ... No / Yes
✓ Add Cypress for both Unit and End-to-End testing? ... No / Yes
✓ Add ESLint for code quality? ... No / Yes
✓ Add Prettier for code formatting? ... No / Yes

Scaffolding project in /Users/chaipeng/Desktop/小兔鲜新版/pro/vue3-project...

Done. Now run:

cd vue3-project
npm install
npm run dev
```

熟悉项目和关键文件



关键文件：

1. vite.config.js - 项目的配置文件 基于vite的配置
2. package.json - 项目包文件 核心依赖项变成了 Vue3.x 和 vite
3. main.js - 入口文件 createApp函数创建应用实例
4. app.vue - 根组件 SFC单文件组件 script - template - style
 - 变化一：脚本script和模板template顺序调整
 - 变化二：模板template不再要求唯一根元素
 - 变化三：脚本script添加setup标识支持组合式API
5. index.html - 单页入口 提供id为app的挂载点

组合式API - setup选项

1. setup选项的写法和执行时机

写法

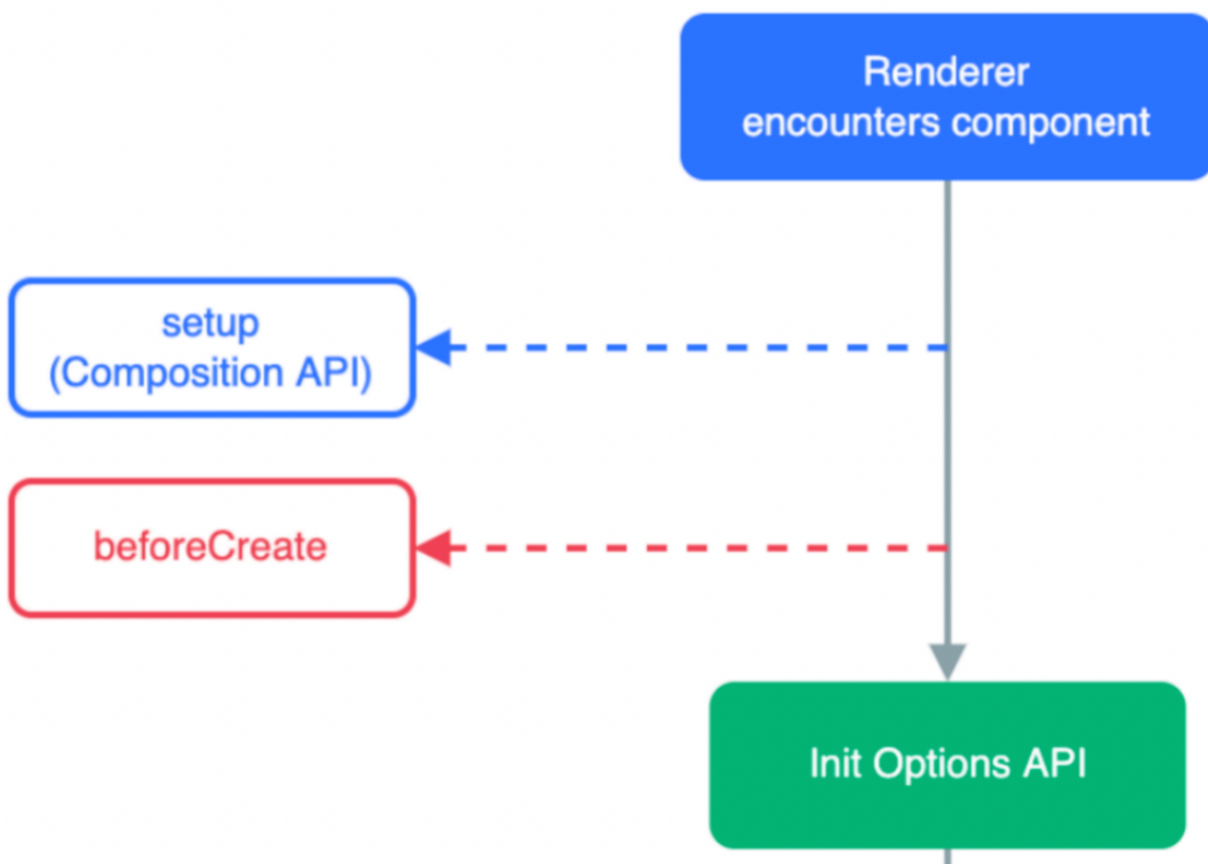
```
<script>
  export default {
    setup(){

    },
    beforeCreate(){

    }
  }
</script>
```

执行时机

在beforeCreate钩子之前执行



2. setup中写代码的特点

在setup函数中写的数据和方法需要在末尾以对象的方式return，才能给模版使用

```
<script>
  export default {
```

```
    setup(){
      const message = 'this is message'
      const logMessage = ()=>{
        console.log(message)
      }
      // 必须return才可以
      return {
        message,
        logMessage
      }
    }
  }
</script>
```

3. <script setup>语法糖

script标签添加 setup标记，不需要再写导出语句，默认会添加导出语句 setup 中的this是undefined,因为这个钩子太早了,比created钩子还早

```
<script setup>
  const message = 'this is message'
  const logMessage = ()=>{
    console.log(message)
  }
</script>
```

组合式API - reactive和ref函数

1. reactive

接受对象类型数据的参数传入并返回一个响应式的对象

```
<script setup>
  // 导入
  import { reactive } from 'vue'
  // 执行函数 传入参数 变量接收
  const state = reactive({
    msg: 'this is msg'
  })
  const setSate = ()=>{
    // 修改数据更新视图
    state.msg = 'this is new msg'
  }
</script>

<template>
  {{ state.msg }}
```

```
<button @click="setState">change msg</button>
</template>
```

2. ref

接收简单类型或者对象类型的数据传入并返回一个响应式的对象

```
<script setup>
// 导入
import { ref } from 'vue'
// 执行函数 传入参数 变量接收
const count = ref(0)
const setCount = ()=>{
  // 修改数据更新视图必须加上.value
  count.value++
}
</script>

<template>
  <button @click="setCount">{{count}}</button>
</template>
```

3. reactive 对比 ref

1. 都是用来生成响应式数据
2. 不同点
 1. reactive不能处理简单类型的数据
 2. ref参数类型支持更好，但是必须通过.value做访问修改
 3. ref函数内部的实现依赖于reactive函数
3. 在实际工作中的推荐
 1. 推荐使用ref函数，减少记忆负担，小兔鲜项目都使用ref

组合式API - computed

计算属性基本思想和Vue2保持一致，组合式API下的计算属性只是修改了API写法 计算属性的结果会被缓存起来，只有当依赖的数据发生变化时才会重新计算

```
<script setup>
// 导入
import {ref, computed } from 'vue'
// 原始数据
const count = ref(0)
// 计算属性
const doubleCount = computed(()=>count.value * 2)

// 原始数据
```

```
const list = ref([1,2,3,4,5,6,7,8])
// 计算属性list
const filterList = computed(item=>item > 2)
</script>
```

组合式API - watch

侦听一个或者多个数据的变化，数据变化时执行回调函数，两个额外参数 immediate控制立刻执行，deep开启深度侦听

1. 侦听单个数据

```
<script setup>
  // 1. 导入watch
  import { ref, watch } from 'vue'
  const count = ref(0)
  // 2. 调用watch 侦听变化
  watch(count, (newValue, oldValue)=>{
    console.log(`count发生了变化, 老值为${oldValue}, 新值为${newValue}`)
  })
</script>
```

2. 侦听多个数据

侦听多个数据，第一个参数可以改写成数组的写法

```
<script setup>
  // 1. 导入watch
  import { ref, watch } from 'vue'
  const count = ref(0)
  const name = ref('cp')
  // 2. 调用watch 侦听变化
  watch([count, name], ([newCount, newName],[oldCount,oldName])=>{
    console.log(`count或者name变化了, [newCount, newName],[oldCount,oldName]`)
  })
</script>
```

3. immediate

在侦听器创建时立即出发回调，响应式数据变化之后继续执行回调

```
<script setup>
  // 1. 导入watch
  import { ref, watch } from 'vue'
  const count = ref(0)
```

```
// 2. 调用watch 侦听变化
watch(count, (newValue, oldValue)=>{
  console.log(`count发生了变化, 老值为${oldValue},新值为${newValue}`)
},{
  immediate: true
})
</script>
```

4. deep

通过watch监听的ref对象默认是浅层侦听的，直接修改嵌套的对象属性不会触发回调执行，需要开启deep

```
<!-- 浅层侦听 -->
<script setup>
  // 1. 导入watch
  import { ref, watch } from 'vue'
  const state = ref({ count: 0 })
  // 2. 监听对象state
  watch(state, ()=>{
    console.log('数据变化了')
  })
  const changeStateByCount = ()=>{
    // 直接修改不会引发回调执行
    // 浅层侦听, 直接修改嵌套属性不会触发回调执行
    state.value.count++
  }
</script>

<!-- 深度监听全部 -->
<script setup>
  // 1. 导入watch
  import { ref, watch } from 'vue'
  const state = ref({ count: 0 })
  // 2. 监听对象state 并开启deep
  // deep:true是对state整体的监听,里面的任何属性变化都会产生回调执行
  watch(state, ()=>{
    console.log('数据变化了')
  },{deep:true})
  const changeStateByCount = ()=>{
    // 此时修改可以触发回调
    state.value.count++
  }
</script>

<!-- 深度监听某个属性 -->
<script setup>
  // 1. 导入watch
  import { ref, watch } from 'vue'
  const state = ref({ count: 0 })
  // 深度监听某个属性
```



```
watch(()=>state.value.count, (newValue, oldValue)=>{
  console.log(`count发生了变化, 老值为${oldValue},新值为${newValue}`)
})
const changeStateByCount = ()=>{
  // 此时修改可以触发回调
  state.value.count++
}
</script>
```

组合式API - 生命周期函数

1. 选项式对比组合式

选项式API	组合式API
beforeCreate/created	setup
beforeMount	onBeforeMount
mounted	onMounted
beforeUpdate	onBeforeUpdate
updated	onUpdated
beforeUnmount	onBeforeUnmount
unmounted	onUnmounted

2. 生命周期函数基本使用

1. 导入生命周期函数

2. 执行生命周期函数, 传入回调

```
<script setup>
import { onMounted } from 'vue'
const getList = () => {
  setTimeout(()=>{
    console.log('获取列表');
  },1000)
}

// 一进页面就获取,类似created
getList()
```

```
//在mounted生命周期中执行
onMounted(()=>{
  console.log('mounted生命周期函数,-逻辑1');
})

//写成函数的调用方式,可以多次调用,不冲突,按照顺序执行
onMounted(()=>{
  console.log('mounted生命周期函数,-逻辑2');
})

</script>
```

3. 执行多次

生命周期函数执行多次的时候，会按照顺序依次执行

```
<script setup>
import { onMounted } from 'vue'
onMounted(()=>{
  // 自定义逻辑
})

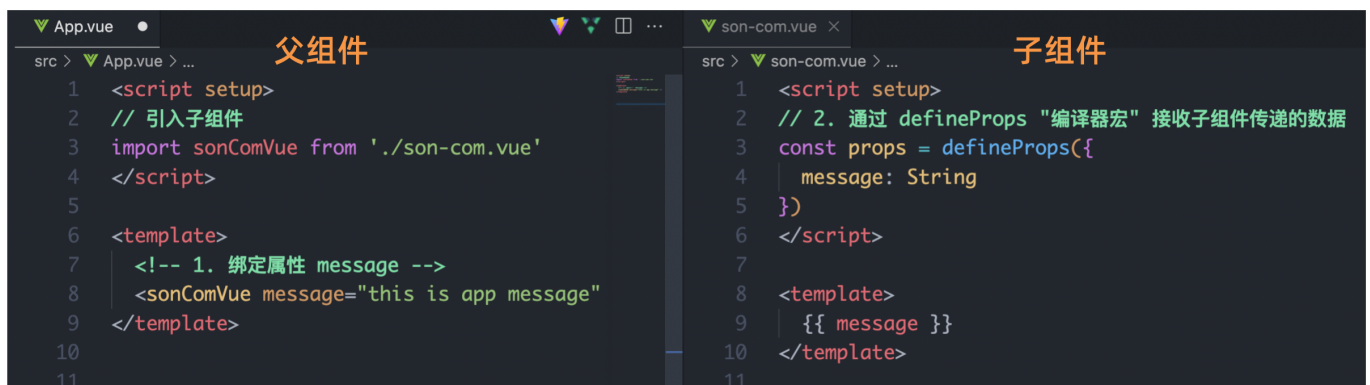
onMounted(()=>{
  // 自定义逻辑
})
</script>
```

组合式API - 父子通信

1. 父传子

基本思想

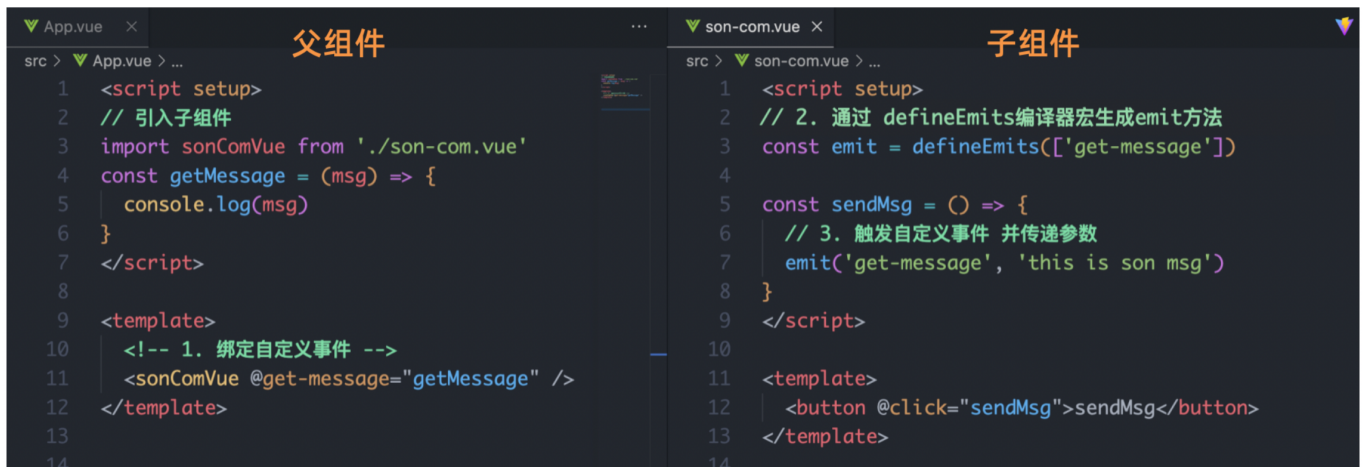
1. 父组件中给子组件绑定属性
2. 子组件内部通过props选项接收数据



2. 子传父

基本思想

1. 父组件中给子组件标签通过@绑定事件
2. 子组件内部通过 emit 方法触发事件



组合式API - 模版引用

概念: 通过 ref标识 获取真实的 dom对象或者组件实例对象 最新版本使用useTemplateRef() 函数获取模版引用

useTemplateRef() 函数, 3.5新增

返回一个浅层 ref, 其值将与模板中的具有匹配 ref attribute 的元素或组件同步。

类型

```
function useTemplateRef<T>(key: string): Readonly<ShallowRef<T | null>>
```

示例

```
<script setup>
import { useTemplateRef, onMounted } from 'vue'

const inputRef = useTemplateRef('input')
//之前是 const inputRef = ref(null)

onMounted(() => {
  inputRef.value.focus()
})
</script>

<template>
```

```
<input ref="input" />
</template>
```

1. 基本使用

实现步骤：

1. 调用ref函数生成一个ref对象
2. 通过ref标识绑定ref对象到标签



```
1  <script setup>
2  import { ref } from 'vue'
3  // 1. 调用ref函数得到ref对象
4  const h1Ref = ref(null)
5  </script>
6
7  <template>
8    <!-- 2. 通过ref标识绑定ref对象 -->
9    <h1 ref="h1Ref">我是dom标签h1</h1>
10 </template>
11
```

2. defineExpose

默认情况下在 `<script setup>` 语法糖下组件内部的属性和方法是不开放给父组件访问的，可以通过 `defineExpose` 编译宏指定哪些属性和方法容许访问 说明：指定 `testMessage` 属性可以被访问到

```
1 <script setup>
2 import { ref } from 'vue'
3 const testMessage = ref('this is test msg')
4 </script>
5
```

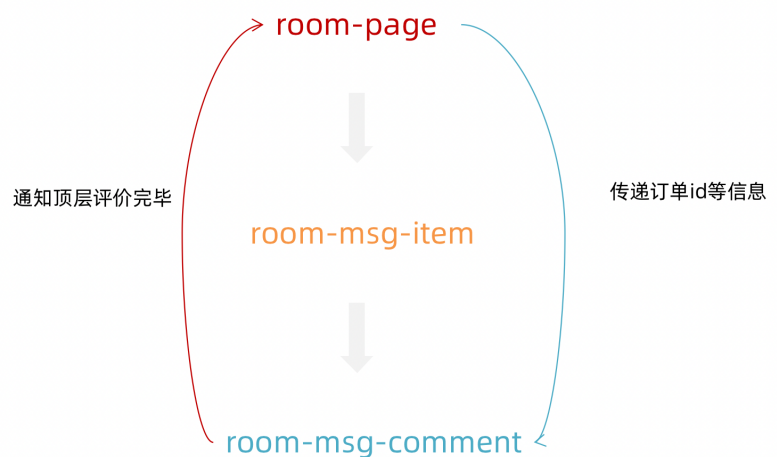


```
1 <script setup>
2 import { ref } from 'vue'
3 const testMessage = ref('this is test msg')
4 defineExpose({
5   testMessage
6 })
7 </script>
```

组合式API - provide和inject

1. 作用和场景

顶层组件向任意的底层组件传递数据和方法，实现跨层组件通信



2. 跨层传递普通数据

实现步骤

1. 顶层组件通过 `provide` 函数提供数据
2. 底层组件通过 `inject` 函数提供数据

顶层组件

```
1 provide('key', 顶层组件中的数据)
```

底层组件

```
1 const message = inject('key')
```

3. 跨层传递响应式数据

在调用provide函数时，第二个参数设置为ref对象

顶层组件

```
1 provide('app-key', ref对象)
```

底层组件

```
1 const message = inject('app-key')
```

4. 跨层传递方法

顶层组件可以向底层组件传递方法，底层组件调用方法修改顶层组件的数据


```
1 const setCount = () => {  
2   count.value++  
3 }  
4  
5 provide('setCount-key', setCount)
```

顶层组件

```
1 const setCount = inject('setCount-key')
```

底层组件

Vue3.3 新特性-defineOptions

背景说明：

有 `<script setup>` 之前，如果要定义 props, emits 可以轻而易举地添加一个与 setup 平级的属性。

但是用了 `<script setup>` 后，就没法这么干了 setup 属性已经没有了，自然无法添加与其平级的属性。

为了解决这一问题，引入了 `defineProps` 与 `defineEmits` 这两个宏。但这只解决了 props 与 emits 这两个属性。

如果我们要定义组件的 name 或其他自定义的属性，还是得回到最原始的用法——再添加一个普通的 `<script>` 标签。

这样就会存在两个 `<script>` 标签。让人无法接受。

所以在 Vue 3.3 中新引入了 `defineOptions` 宏。顾名思义，主要是用来定义 Options API 的选项。可以用 `defineOptions` 定义任意的选项， props, emits, expose, slots 除外（因为这些可以使用 `defineXXX` 来做到）

```
1 <script setup>  
2 defineOptions({  
3   name: 'Foo',  
4   inheritAttrs: false,  
5   // ... 更多自定义属性  
6 })  
7 </script>
```

vue 复制代码

Vue3.3新特性-defineModel

在Vue3中，自定义组件上使用`v-model`时，相当于传递一个`modelValue`属性，同时触发`update:modelValue`事件

```
1 <Child v-model="isVisible">
2 // 相当于
3 <Child :modelValue="isVisible" @update:modelValue="isVisible=$event">
```

我们需要先定义 `props`，再定义 `emits`。其中有许多重复的代码。如果需要修改此值，还需要手动调用 `emit` 函数。

于是乎 `defineModel` 诞生了。

```
1 <script setup>
2 const modelValue = defineModel()
3 modelValue.value++
4 </script>
```

生效需要配置 `vite.config.js` 文件

```
import { fileURLToPath, URL } from 'node:url'

import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'

// https://vitejs.dev/config/
export default defineConfig({
  plugins: [
    vue({
      script: {
        defineModel: true
      }
    })
  ],
  resolve: {
    alias: {
      '@': fileURLToPath(new URL('./src', import.meta.url))
    }
  }
})
```

Vue3 状态管理 - Pinia

1. 什么是Pinia

Pinia 是 Vue 的专属的最新状态管理库，是 Vuex 状态管理工具的替代品



1. 提供更加简单的API（去掉了 mutation）
2. 提供符合组合式风格的API（和 Vue3 新语法统一）
3. 去掉了 modules 的概念，每一个 store 都是一个独立的模块
4. 配合 TypeScript 更加友好，提供可靠的类型推断

2. 手动添加Pinia到Vue项目

后面在实际开发项目的时候，Pinia可以在项目创建时自动添加，现在我们初次学习，从零开始：

1. 使用 Vite 创建一个空的 Vue3项目

```
npm init vite@latest
```

1. 按照官方文档安装 pinia 到项目中

3. Pinia基础使用

1. 定义store
2. 组件使用store

定义Store（state + action）

```
1 import { defineStore } from 'pinia'
2 import { ref } from 'vue'
3 export const useCounterStore = defineStore('counter', () => {
4   // 数据 (state)
5   const count = ref(0)
6
7   // 修改数据的方法 (action)
8   const increment = () => {
9     count.value++
10  }
11
12  // 以对象形式返回
13  return { count, increment }
14 })
```



组件使用Store

```
1 <script setup>
2 // 1. 导入 `useCounterStore` 方法
3 import { useCounterStore } from '@/stores/counter'
4 // 2. 执行方法得到counterStore对象
5 const counterStore = useCounterStore()
6 </script>
7
8 <template>
9   <button @click="counterStore.increment">
10     {{ counterStore.count }}
11   </button>
12 </template>
13
```

4. getters实现

Pinia中的 getters 直接使用 computed函数 进行模拟, 组件中需要使用需要把 getters return出去



```
1 // 数据 (state)
2 const count = ref(0)
3
4 // getter
5 const doubleCount = computed(() => count.value * 2)
```

```
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'

// 1. 定义 Store
// 组合式api里, 引入外部的js方法一般用use开头
export const useCounterStore = defineStore('counter', () => {
  const count = ref(99)
  const add = () => {
    count.value++
  }
  const sub = () => {
    count.value--
  }

  // 3. 计算属性getters(computed)
  const double = computed(() => {
    return count.value * 2
  })

  const msg = ref('hello counter')

  return {
    count,
    msg,
    add,
    sub,
    double
  }
})
```

5. action异步实现

方式：异步action函数的写法和组件中获取异步数据的写法完全一致

- 接口地址：http://geek.itheima.net/v1_0/channels
- 请求方式：get

- 请求参数: 无

```
1 // 异步action
2 const getList = async ()=>{
3   const res = await axios.request<接口数据类型>({})
4 }
```

需求: 在Pinia中获取频道列表数据并把数据渲染App组件的模板中

object {2}

▼ data

object {1}

▼ channels

array[object] {2}

id

integer

name

string

message

string

```
<script setup>
import Son1Com from './components/Son1Com.vue'
import Son2Com from './components/Son2Com.vue'
import { useCounterStore } from './store/counter.js'
import { useChannelStore } from './store/channel.js'

const counterStore = useCounterStore()
console.log(counterStore);

const channelStore = useChannelStore()

</script>

<template>
  <div><h3>App.vue根组件
    -- {{ counterStore.count }}
  </div>
</template>
```

```
-- {{ counterStore.msg }}
</h3></div>
<Son1Com />
<Son2Com />
{{ counterStore.double }}
<button @click="channelStore.getList">获取频道数据</button>
<ol v-if="channelStore.channelList.length > 0">
  <li v-for="item in channelStore.channelList">{{ item.name }}</li>
</ol>
</template>

<style scoped>

</style>
```

6. storeToRefs工具函数

直接解构store对象会丢失响应式,可以使用storeToRefs函数进行解构 方法可以直接解构,数据需要通过storeToRefs函数进行解构 storeToRefs函数的本质是使用Vue的toRefs函数对store对象进行解构

使用storeToRefs函数可以辅助保持数据 (state + getter) 的响应式解构



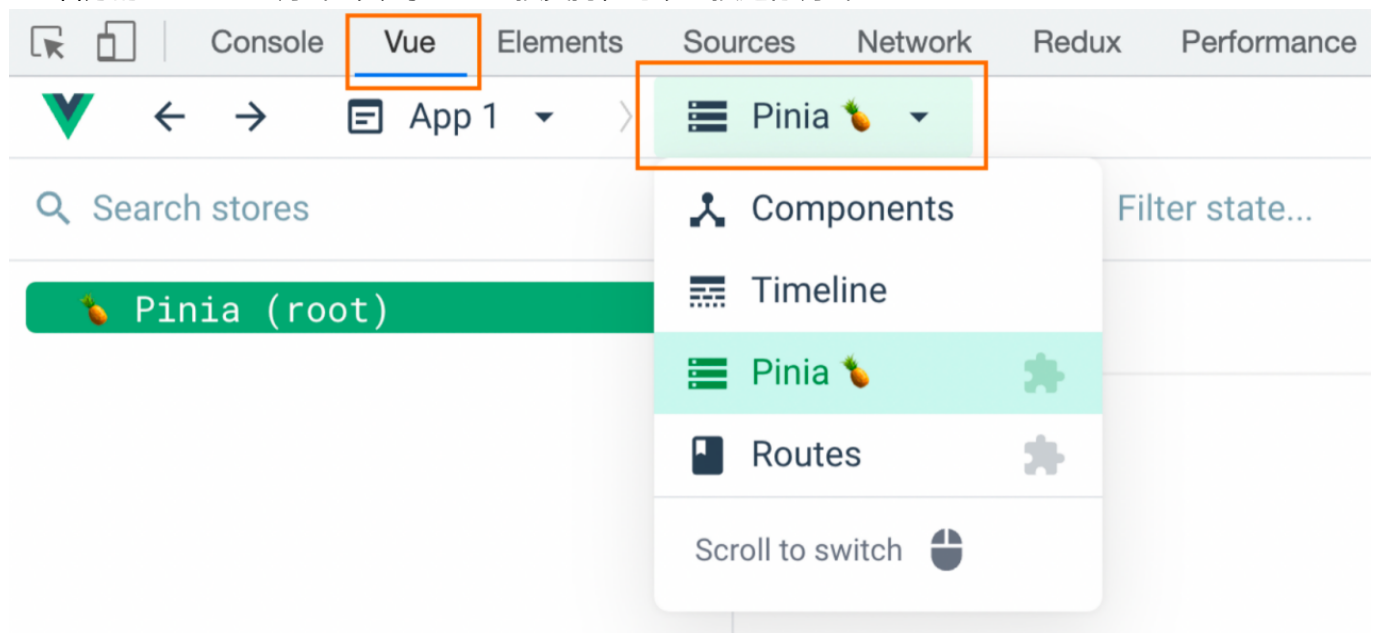
```
1 // 响应式丢失 视图不再更新
2 const { count, doubleCount } = counterStore
```



```
1 // 保持数据响应式
2 const { count, doubleCount } = storeToRefs(counterStore)
```

7. Pinia的调试

Vue官方的 dev-tools 调试工具 对 Pinia直接支持，可以直接进行调试



8. Pinia持久化插件

官方文档: <https://prazdevs.github.io/pinia-plugin-persistedstate/zh/>

1. 安装插件 pinia-plugin-persistedstate

```
npm i pinia-plugin-persistedstate
```

1. 使用 main.js

```
import persist from 'pinia-plugin-persistedstate'
...
app.use(createPinia().use(persist))
```

1. 配置 store/counter.js

```
import { defineStore } from 'pinia'
import { computed, ref } from 'vue'

export const useCounterStore = defineStore('counter', () => {
  ...
  return {
    count,
    doubleCount,
    increment
  }
}, {
  persist: true
})
```

1. 其他配置，看官网文档即可

```
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'

// 1. 定义 Store
export const useCounterStore = defineStore('counter', () => {
  const count = ref(99)
  const add = () => {
    count.value++
  }
  const sub = () => {
    count.value--
  }

  // 3. 计算属性getters(computed)
  const double = computed(() => {
    return count.value * 2
  })

  const msg = ref('hello counter')

  return {
    count,
    msg,
    add,
    sub,
    double
  }
}, {
  // persist: true //开启数据持久化
  persist: {
    key: 'counter', //指定存储的key,默认是pinia
    storage: localStorage, //指定存储方式,默认是localStorage
    paths: ['count'] //指定需要持久化的状态属性
  }
})
```